

Логічні вирази. Галуження

Логічний тип, логічні оператори та логічні вирази

Логічний тип (bool) складається з двох взаємовиключних значень True і False¹. У Python логічний тип є підмножиною типу int, оскільки його значення поведуться так само, як цілі числа 1 і 0, лишень відображаються як True і False для кращого сприйняття:

```
print(3*True + False - 1)          # 2
```

Прості логічні вирази зазвичай утворюються з використанням операторів порівняння: == ("дорівнює"), != ("не дорівнює"), < ("менше"), > ("більше"), <= ("менше або дорівнює"), >= ("більше або дорівнює"). Наприклад (після логічного виразу вказано його значення):

```
3 == 5          # False
3 != 3.0        # False
5 >= 5          # True
```

Зверніть увагу, що порівняння дійсних чисел, через їх наближене подання у пам'яті комп'ютера, може видати неочікувані результати:

```
print(0.1**3 == 0.001)            # False
```

Певним чином виправити цю ситуацію можна, встановлюючи близькість між числами з потрібною точністю (але в межах точності представлення типу float):

```
x, y = 0.1**3, 0.001
print(abs(x-y) < 10e-5)           # True
```

Порівнювати можна не тільки числа, а рядки, списки, кортежі та множини.

Порівняння значень різних типів у загальному випадку не допускається — спроба порівняти число і рядок викличе помилку:

```
5 > '2'          # TypeError: '>' not supported between instances of 'int' and 'str'
```

Але:

¹ Великі літери важливі, інакше інтерпретатор трактуватиме true і false як імена змінних

```
5 != '5'      # True (справді, число 5 і символ '5' – не одне й те саме)
```

З операторів порівняння можна утворювати ланцюжки:

```
1 != 2 <= 2+1 > 1.5      # True
'A' < 'B' > 'C'          # False
```

Для об'єднання результатів перевірок використовуються логічні оператори:

```
not (2 > 3)               # True
2 > 3 or 3 < 4            # True
2 + 2 == 4 and 2 * 2 != 4 # False
```

Логічні вирази можуть використовувати оператор перевірки на входження in, який використовується для перевірки належності об'єкта певному складеному типу:

```
'u' in 'Ukraine'         # False
'Ukr' in 'Ukraine'       # True
'u' not in 'Ukraine'     # True
```

Інструкція if

Умовна інструкція if є складеною інструкцією (тобто, містить після символу ":" блоки інших інструкцій з відступами) і має такий загальний синтаксис:

```
if <логічний вираз 1>:
    <блок інструкцій 1>
elif <логічний вираз 2>:      # необов'язкова частина (elif може бути декілька)
    <блок інструкцій 2>
else:                        # необов'язкова частина
    <блок інструкцій 3>
```

Спочатку інтерпретатор перевіряє умову, задану логічним виразом у блоці if. Якщо ця умова виконується, то виконується вкладені інструкції цього блоку. Якщо ні — по черзі перевіряються умови усіх блоків elif. При цьому виконуються вкладені інструкції блоку, умова у якому задовільнилася першою. Якщо жодна з умов у блоках elif не виконалася, виконуються вкладені інструкції блоку else. Нижче наведені приклади різних варіантів інструкції if...elif...else.

```
word = 'інформатика'
if input('Введіть літеру: ') in word:
    print('Введена Вами літера є у слові, word')
```

```

number = int(input('Введіть число: '))
if number % 2:
    print('Число', number, 'непарне')
else:
    print('Число', number, 'парне')

x, y = input('Введіть перше число: '), input('Введіть друге число: ')
if x > y:
    print('Перше число більше за друге')
elif x < y:
    print('Перше число менше за друге')
else:
    print('Числа однакові')

```

Відступи (пробіли або символи табуляції на початку рядка) є частиною синтаксису Python. Інтерпретатор автоматично визначає межі блоків складених інструкцій за величиною відступів. Усі інструкції в межах одного блоку повинні мати однаковий відступ, який має бути оформлений або пробілами, або символами табуляції (змішування не допускається).

Оскільки складені інструкції можуть містити в собі інші складені інструкції, то неправильне оформлення відступів може призвести до некоректних результатів (семантичні помилки). Наприклад, синтаксично правильний код

```

age = int(input('Введіть Ваш вік: '))
if age < 18:
    print('Ви неповнолітні')
    if age < 5:
        print('Ви не школяр')
    else:
        print('Ви повнолітні')

```

видасть некоректний результат для віку 10 років: буде надруковано як 'Ви неповнолітні', так і 'Ви повнолітні'. Справа в тому, що інструкція else стосується другого if, а не першого. Правильний варіант:

```

age = int(input('Введіть Ваш вік: '))
if age < 18:
    print('Ви неповнолітні')
    if age < 6:
        print('Ви не школяр')
else:
    print('Ви повнолітні')

```

В окремих випадках допускається записувати умовну інструкцію в один рядок:

```

if hours > 24: print('Пройшло більше доби')

```

або у вигляді тернарного оператора

```
x = 1 if y>3 or y!=5 else 0
```

Остання інструкція є еквівалентом складеної інструкції

```
if y>3 and y!=5:  
    x = 1  
else:  
    x = 0
```

Безоператорні логічні вирази

Логічні вирази не обов'язково мають містити оператори порівняння, логічні оператори чи оператор перевірки на входження. Будь який об'єкт (число, рядок, функція тощо) трактується як логічний вираз, якщо він входить в умовну інструкцію, при цьому відмінні від нуля числа і непорожні об'єкти завжди інтерпретуються як True, а нуль, порожні об'єкти та спеціальний об'єкт None — як False.

Інструкція match

У версії Python 3.10 з'явилася інструкція match, яка дає можливість зіставити значення виразу з певним шаблоном. Вона чимось схожа на інструкцію if/else/elif, але має значно ширший функціонал, даючи змогу виймати дані зі складених типів та застосовувати дії до частин об'єктів².

```
def screamer(beginning):  
    match beginning:  
        case "Слава Україні!":  
            print("Героям слава!")  
        case "Слава нації!":  
            print("Смерть ворогам!")  
        case "Україна!":  
            print("Понад усе!")  
        case _:  
            print("Путін - ...")
```

² Розширені можливості інструкції match не виносяться для вивчення - за бажанням Ви можете ознайомитися з ними у документації чи інших відкритих джерелах